Major Project Report

EduSphere

Project Category: University Based Project

Projexa Team Id- 26E3142

Submitted in partial fulfilment of the requirement of the degree of

BACHELOR OF TECHNOLOGY (CSE)

Semester-VI

to

K.R Mangalam University

by

Adeela Azeez (2301010282) Section-E
Vaibhav Vaid (2301010289) Section-E
Ashutosh Singh(2301010293) Section-E
Satdev (2301010298) Section-E
Himani Tyagi(2301010300) Section-E

Under the supervision of

Ms. Iflah Aijaz
Assisstant Professor

Department of Computer Science and Engineering
School of Engineering and Technology
K.R Mangalam University, Gurugram- 122001, India

May 2026

DECLARATION

I hereby declare that this project report is my original work and has not been submitted elsewhere. The information presented here is true and correct to the best of my knowledge.

(Signature)

Vaibhav Vaid & Adeela Azeez

2301010289 / 2301010282

Date: 06/04/2026

INDEX

S No	Content	Page No
1.	Project Completion Certificate	1
2.	Abstract	2
3.	Introduction	3
4.	Motivation	4
5.	Literature Review	5-6
6.	Gap Analysis	7
7.	Problem Statement	8
8.	Objectives	9
9.	Tools/platform Used	10
10.	Methodology	11-14
11.	Experimental Setup	15-16
12.	Evaluation Metrics	17-18
13.	Results And Discussion	19-20
14.	Conclusion & Future Work	21-22
15.	References	23
16.	Annexure I: Plagiarism Declaration Certificate	24
17.	Annexure II: Plagiarism Report	25
18.	Annexure III: Implementation code	26-29

CERTIFICATE OF UNIVERSITY-BASED PROJECT COMPLETION

This is to certify that the students listed below have **successfully completed** a project based on a **University-based problem**, undertaken as part of their academic or innovation-related engagement at **K. R. Mangalam University**.

Title of the Project:
EduSphere

Name(s) of Student(s):

- Vaibhav Vaid– Roll No: 2301010289
- Adeela Azeez– Roll No: 2301010282
- Ashutosh Singh – Roll No: 2301010293
- Satdev – Roll No: 2301010298
- Himani Tyagi – Roll No: 2301010300

Duration:
From **January 2026** to **May 2026**

This is to acknowledge that the above-mentioned project has been **successfully completed** and has effectively addressed a **real-world problem within the university ecosystem**.
The initiative is appreciated for its **innovation, practical relevance, and contribution** toward institutional improvement and **student-led problem solving**.

We congratulate the project team for their dedication, creativity, and successful completion of this university-based initiative.

Signature: _____
Ms. Iflah Aijaz

Date: 06/04/2026

School of Engineering & Technology

K. R. Mangalam University

2.ABSTRACT

Universities manage a wide range of academic, administrative, and student support activities on a daily basis. However, these services are often handled through multiple independent systems or manual processes, leading to delays, lack of transparency, and increased workload for both students and staff. The absence of a centralized platform makes it difficult to track student requests, maintain records, and ensure efficient communication.

This project proposes the development of a Unified University Management & Student Support Platform, a web-based system that integrates academic management and student support services into a single digital solution. The platform provides role-based access for students, faculty, and administrators, allowing them to perform their respective tasks efficiently. By automating key processes and enabling real-time communication, the system aims to improve service delivery, reduce manual effort, and enhance the overall university experience.

3. Introduction

With the rapid growth of higher education institutions, managing academic operations and student services has become increasingly complex. Universities are required to handle large volumes of data related to students, faculty, examinations, attendance, scholarships, internships, and grievance redressal. Traditionally, these activities are managed using separate software tools, emails, notice boards, or physical office visits.

Such fragmented systems often result in poor coordination, delayed responses, and difficulty in maintaining accurate records. Students face challenges in accessing timely information, while faculty and administrators struggle with manual tracking and repetitive tasks. In the era of digital transformation, there is a strong need for an integrated and reliable system that can streamline university operations.

The Unified University Management & Student Support Platform focuses on addressing these issues by offering a centralized web application that simplifies academic management and improves communication between all stakeholders. The system aims to support efficient decision-making, enhance transparency, and provide a structured approach to managing university processes.

4.MOTIVATION

The rapid growth in the number of students and academic activities in universities has made traditional academic management systems increasingly inefficient. Many universities still rely on a combination of manual record-keeping, spreadsheets, emails, and disconnected web portals to manage academic operations. This often results in data redundancy, delayed communication, lack of transparency, and increased administrative workload.

Students frequently face difficulties in accessing accurate and up-to-date academic information such as schedules, notices, attendance records, and academic progress. Faculty members spend significant time managing student data manually, which reduces the time available for teaching and research. Administrators, on the other hand, struggle to maintain consistency and coordination across departments due to the absence of a unified system.

The motivation behind this project is to address these challenges by developing a centralized digital platform that simplifies academic processes and improves overall efficiency. The project is also motivated by the growing adoption of digital solutions in education and the need for scalable, secure, and user-friendly systems. By providing a single platform for academic services, the proposed system aims to enhance user experience, reduce operational overhead, and support the digital transformation of universities.

5. LITERATURE REVIEW

Several studies and existing systems highlight the importance of digital platforms in managing academic and administrative operations within universities. Learning Management Systems (LMS) such as Moodle and Google Classroom focus primarily on course delivery and learning resources, but they offer limited support for administrative and institutional data management. Similarly, enterprise resource planning (ERP) systems used by universities provide comprehensive administrative features but are often expensive, complex, and difficult to customize.

Research papers on university information systems emphasize the benefits of centralized data management, improved communication, and automation of academic workflows. Many studies report that integrated academic platforms lead to better decision-making, reduced human error, and enhanced transparency. However, literature also points out challenges such as poor usability, lack of real-time updates, and limited integration between academic and administrative modules.

Existing student portals often provide basic functionality such as viewing results or notices but fail to offer a holistic academic experience. Furthermore, some systems are designed without considering the diverse needs of students, faculty, and administrators, leading to low adoption rates. These findings indicate that while digital solutions exist, there is still a need for a flexible, integrated, and user-centric platform that effectively addresses the academic management requirements of modern universities.

LITERATURE REVIEW TABLE

S. No.	Author(s) & Year	Title	Methodology / Focus	Key Findings	Research Gap Identified
1	Alalwan et al. (2017)	Factors influencing adoption of mobile learning in higher education	Empirical study on technology adoption	User-friendly interfaces and perceived usefulness significantly affect adoption	Limited focus on integrated academic management systems
2	Altbach et al. (2019)	Trends in global higher education	Analytical study on higher education systems	Highlights the need for digital transformation in universities	Does not provide a unified digital platform solution
3	Bates (2018)	Teaching in a digital age	Conceptual framework for digital learning environments	Emphasizes structured digital systems for academic support	Lacks implementation of administrative workflow automation
4	DeLone & McLean (2003)	Information systems success model	Theoretical IS success model	System quality, information quality, and user satisfaction are key success factors	Model not applied to university-wide management platforms
5	Ellis & Goodyear (2016)	Integrating digital and physical learning environments	Case studies in higher education	Digital platforms improve communication and transparency	Limited discussion on role-based access systems
6	Kumar & Sharma (2020)	Digital platforms in higher education administration	Survey-based research	Digital systems reduce administrative workload and delays	No unified solution for students, faculty, and admins
7	Ozkan & Koseler (2009)	Evaluation of e-learning systems	Multi-dimensional evaluation model	System usability and content quality affect system success	Focused only on learning systems, not administrative services
8	Sarker et al. (2013)	Qualitative studies in information systems	Methodological review	User-centered design improves system effectiveness	Lacks application to academic support platforms

6. GAP ANALYSIS

A gap analysis is conducted to identify the shortcomings of the existing university management systems and to justify the need for the proposed solution.

Existing System Limitations

- Academic management and student support services are handled using multiple independent systems or manual processes.
- Lack of system integration leads to fragmented and inconsistent academic data.
- Students rely on notice boards, emails, and physical office visits for accessing academic information.
- No centralized platform exists for tracking attendance, results, grievances, and notifications.
- Grievance handling is manual and time-consuming, with no proper tracking mechanism.
- Faculty members face difficulty in monitoring student performance and managing academic records.
- Administrators experience challenges in workflow management and report generation.
- Absence of role-based access control increases security and data privacy risks.
- Manual data handling increases the chances of human errors and data redundancy.

7. PROBLEM STATEMENT

In most universities, academic management, student support services, and administrative processes are handled through multiple independent systems or informal communication channels. Students rely on notice boards, emails, messaging applications, and physical visits to administrative offices to access information related to attendance, examinations, results, scholarships, internships, mentoring, and grievance redressal. This fragmented approach leads to scattered information, inconsistent records, and delays in communication.

Due to the absence of a centralized platform, students often face difficulties in tracking their academic progress, receiving timely notifications, and following up on grievances or requests. Faculty members and administrators also experience challenges in managing attendance, monitoring student performance, responding to student queries, and maintaining accurate academic records. Manual or semi-automated processes increase the workload, introduce human errors, and reduce overall operational efficiency.

Furthermore, the lack of transparency in existing systems results in repeated queries, miscommunication, and dissatisfaction among students. Administrators struggle to generate consolidated reports, manage workflows efficiently, and ensure data security across different departments. The absence of role-based access and secure authentication mechanisms further raises concerns regarding data integrity and privacy.

Therefore, there is a critical need for a unified, secure, and digital platform that integrates academic management and student support services into a single system. Such a platform should provide role-based access for students, faculty, and administrators, ensure real-time information availability, improve transparency, streamline workflows, and enhance the overall efficiency of university operations.

8. OBJECTIVES

- To establish a centralised digital platform for coordinating academic, administrative, and student support services
- To get rid of disjointed systems such as manual office visits, emails, and notice boards
- To make information about attendance, tests, results, scholarships, internships, and mentoring easy access
- To increase transparency and communication among the administration, instructors, and students
- To facilitate the effective tracking and management of student complaints and requests
- To lessen the administrative burden, repetitive enquiries, and delays
- To keep secure and organised academic records
- To improve institutional effectiveness and the overall student experience
- To assist university officials in making data-driven decisions.

9. Tools/Technologies Used

To ensure a smooth and efficient development process, the following tools and technologies will be utilized in the development of EduSphere: Unified University Management & Student Support Platform:

- **Front-end:**

React.js will be used to develop interactive, responsive, and role-based user interfaces for students, faculty members, and administrators.

- **Back-end:**

Node.js with Express will be employed to handle server-side logic, authentication, role-based access control, and RESTful API development.

- **Database:**

MongoDB or PostgreSQL will be used to securely store user information, academic records, attendance data, grievances, and notifications.

- **Security & Authentication:**

JWT (JSON Web Tokens) will be implemented for secure authentication, along with Role-Based Access Control (RBAC) to restrict access based on user roles.

- **UI/UX Design:**

Figma will be used for wireframing and prototyping role-based dashboards and user workflows prior to implementation.

- **Deployment & Environment Management:**

The application will be deployed using local servers or Docker-based containerization for academic demonstration purposes. Cloud hosting is considered as part of future enhancements.

10. METHODOLOGY

The development of the **EduSphere: Unified University Management & Student Support Platform** will follow an **iterative and user-centered development methodology.** The project will be executed in structured phases to ensure usability, security, and scalability. The key phases include:

•Research and Analysis:

Study existing university management systems and identify challenges faced by students, faculty, and administrators in managing academic and administrative processes. Inputs will be gathered through discussions, observations, and analysis of current institutional workflows.

•Requirement Specification:

Define detailed functional and non-functional requirements for the Student, Faculty, and Admin modules based on the identified academic and administrative needs.

•Design and Prototyping:

Create system architecture diagrams, database schemas, and UI wireframes using tools such as Figma to visualize role-based dashboards and user interactions.

•Development:

Implement the platform using modern front-end and back-end technologies. Secure authentication, role-based access control (RBAC), and scalable RESTful APIs will be developed to ensure data integrity and system security.

•Testing and Validation:

Perform unit testing, integration testing, and usability testing to identify bugs, verify functionality, and validate system performance across all modules.

•Deployment and Maintenance:

Deploy the application using local or containerized environments (Docker). The system will be maintained and refined based on testing outcomes and future academic requirements.

The most effective methodology for a complex, multi-role system like EduSphere is the **Modular-Iterative Development Lifecycle**. This approach ensures that individual workflows (like Attendance or Gatepass) can be developed, tested, and integrated without disrupting the core system.

1. Core Development Strategy: Modular-Iterative

EduSphere is developed using a **Modular-Iterative approach**. Instead of building the entire system at once, the project is broken down into specific "Business Modules" (Gatepass, Attendance, etc.). Each module undergoes its own mini-lifecycle of design, development, and testing.

Key Characteristics:

- **Separation of Concerns:** The Frontend (React), Backend (Node/Django), and Database (MongoDB/Postgres) are treated as distinct layers.
- **Parallel Workstreams:** The DevOps team handles Docker/Deployment while the developers focus on RBAC and Business Logic.
- **Role-Centric Design:** Every feature is developed specifically through the lens of the five user personas (Student, Faculty, Admin, Parent, Warden).

2. Detailed Methodological Phases

Phase I: Logic & Schema Design (The "Model" Layer)

Before coding the UI, the fundamental data structures are established.

- **ER Modeling:** Creating relationships for the **Multi-level Approval System** (e.g., linking a Student's Gatepass to both a Parent and a Warden).
- **State Management Design:** Defining the "States" of a request (e.g., Pending_Parent -> Pending_Warden -> Approved).

Phase II: Security Framework Development (The "Middleware" Layer)

The backbone of the workflow is the **Security Infrastructure**, which must be established before functional modules are built.

- **JWT Implementation:** Developing the logic to issue and verify tokens.
- **RBAC Mapping:** Creating a permission matrix.
    - *Example:* Defining that only Role: Faculty can hit the POST /api/attendance endpoint.

Phase III: API-Driven Development (The "Controller" Layer)

We follow a **RESTful API methodology** to ensure the frontend and backend remain decoupled.

- **Endpoint Construction:** Building controllers for each workflow (Authentication, Attendance, etc.).
- **Business Logic Integration:** Implementing the "Cron Jobs" mentioned in your workflow to automate threshold alerts for low attendance.

Phase IV: Component-Based UI Development (The "Frontend" Layer)

Using **React.js**, the UI is built as a series of reusable components.

- **Dashboard Personalization:** Conditional rendering is used to display different tools based on the user's role.
- **Stateful UI:** Using React hooks to manage the "Login State" and store the JWT in localStorage.

Phase V: Integration & System Validation

The final phase focuses on the **Full-Stack Handshake**.

- **End-to-End Testing:** Simulating a complete workflow, such as a student submitting a grievance and an Admin responding to it.
- **Deployment Methodology:** Utilizing **Docker** to containerize the environment, ensuring that the middleware and database configurations remain consistent across different servers.

3. Workflow Implementation Mapping

Workflow	**Methodological Focus**	**Key Technical Implementation**
Authentication	Security First	JWT generation + LocalStorage management.
Attendance	Automation	Scheduled Cron Jobs + Threshold logic.
Gatepass	Sequence Logic	Multi-level state transitions (Parent $\rightarrow$ Warden).
Grievance	Interactivity	Chatbot UI integration + CRUD operations.

11. EXPERIMENTAL SETUP

The **Experimental Setup** section describes the specific technical environment, tools, and configurations used to build and validate the **EduSphere** platform. This setup ensures that the system can be replicated and that it functions reliably across different university workflows.

1. Hardware Configuration

The development and testing were conducted on a workstation meeting the following specifications to handle concurrent frontend and backend processes:

- **Processor:** Intel Core i5 (or higher) to manage multiple runtime environments.
- **Memory (RAM):** Minimum 8GB to support the React development server, Backend API, and Database instances simultaneously.
- **Storage:** 256GB SSD for high-speed data indexing and rapid I/O operations during build processes.

2. Software & Development Environment

The platform follows a modern full-stack architecture, utilizing specific frameworks to ensure modularity and security.

- **Frontend Framework: React.js (Vite)** for a fast, component-based user interface, styled with **Tailwind CSS** for responsive design.
- **Backend Runtime: Node.js (Express)** or **Django**, providing a RESTful API architecture to handle business logic.
- **Database Management: PostgreSQL** (Relational) or **MongoDB** (NoSQL) for storing role-based data, logs, and student records.
- **Security Protocol: JWT (JSON Web Tokens)** implemented for stateless, secure authentication across all role-based dashboards.

3. Development Tools & DevOps

To maintain code integrity and streamline the workflow, the following utility suite was employed:

- **IDE: Visual Studio Code** for script writing and debugging.
- **Version Control: GitHub** for collaborative development and code history management.
- **API Testing: Postman** for isolating and validating backend endpoints before frontend integration.
- **Containerization (Optional): Docker** for packaging the application to ensure consistency between development and production environments.

4. Testing & Validation Setup

The experimental validation was performed by simulating a live university ecosystem using the following methods:

- **Multi-Role Simulation:** Five distinct user profiles (**Student, Faculty, Admin, Parent, Warden**) were configured with unique access permissions using **RBAC (Role-Based Access Control)** logic.
- **Data Mocking:** Artificial datasets were generated for:
    - **Attendance:** Simulating daily logs and threshold-based alerts.
    - **Gatepass:** Testing the sequential approval chain from Parent to Warden.
    - **Grievances:** Validating the ticket submission and resolution flow.
- **Environment Compatibility:** **API Validation:** Stress-testing endpoints via Postman for status codes (200 OK, 401 Unauthorized, etc.).
    - **Cross-Browser Testing:** Verified UI stability and responsiveness on **Google Chrome** and **Microsoft Edge**.

12. Evaluation Metrics

The **Evaluation Metrics** section defines the quantitative and qualitative standards used to measure the success of the **EduSphere** platform. These metrics ensure that the system is not only functional but also secure, efficient, and user-centric within a high-pressure university environment.

1. Functional Correctness

This metric validates that every feature performs according to the **Software Requirement Specification (SRS)** without logic errors.

- **Module Integrity:** Verification that the **Attendance tracking** correctly calculates percentages and that the **Gatepass workflow** follows the mandatory sequence (Student -> Parent -> Warden).
- **Role-Specific Logic:** Ensuring the **Admin Dashboard** displays global analytics while the **Student Dashboard** only shows individual records.
- **Success Rate:** Measured by the percentage of successful transactions (e.g., successful grievance submissions vs. total attempts).

2. Performance & Efficiency

Performance metrics focus on the "speed" and "responsiveness" of the system under active use.

- **API Latency:** Measuring the time taken for a **REST API** request (e.g., `GET /api/attendance`) to return a response. Target thresholds are typically $< 200\text{ms}$ for high-performance systems.
- **Bulk Operation Handling:** Evaluating the system's ability to process large datasets, such as a Faculty member marking attendance for 60+ students in a single click without server timeouts.
- **Resource Utilization:** Monitoring CPU and RAM usage on the server-side (Node.js/Django) during peak request periods.

3. Usability & User Experience (UX)

Usability measures how easily a non-technical user (e.g., a Parent or Warden) can navigate the system.

- **Interface Intuition:** Evaluating the "learning curve" of the **React-based UI**—specifically how quickly a user can find the "Submit Gatepass" or "View Marks" buttons.
- **Device Responsiveness:** Using CSS Breakpoints to ensure the **Tailwind CSS** layout remains functional on mobile phones (for Students/Parents) and desktops (for Admins).
- **Navigation Efficiency:** Counting the number of clicks required to complete a task (e.g., a "Three-Click Rule" for reaching any major module).

4. Security & Data Integrity

Security is a critical metric for university management where sensitive student data is involved.

- **Authentication Strength:** Validating that **JWT (JSON Web Tokens)** are issued correctly and expire after a set duration to prevent unauthorized session hijacking.
- **Authorization (RBAC):** Stress-testing the **Role-Based Access Control** to ensure a Student cannot bypass frontend restrictions to access Admin API endpoints.

- **Data Encryption:** Confirming that passwords and sensitive credentials are hashed (e.g., using **Bcrypt**) before being stored in the database.

5. Reliability & Consistency

Reliability ensures the system provides a stable and "truthful" state of data at all times.

- **ACID Compliance:** Ensuring that if a database operation fails (e.g., during a gatepass approval), the system rolls back to a safe state to prevent "ghost" approvals.
- **Workflow Validation:** Confirming that the multi-level approval chain cannot be broken for instance, a Warden should not be able to approve a gatepass if the Parent has already rejected it.

6. Scalability & Extensibility

This metric looks at the future-proofing of the EduSphere architecture.

- **Load Scalability:** The ability of the **Node.js/Django** backend to handle an increasing number of concurrent users as the university grows.
- **Functional Scalability:** How easily a new module (e.g., "Library Management" or "Fee Payment") can be integrated into the existing modular architecture without refactoring the core codebase.

13. Results And Discussions

This section synthesizes the development journey of **EduSphere**, moving from the technical implementation to the tangible outcomes achieved. The **Results and Discussion** phase evaluates how the integration of modern web stacks (React, Node.js/Django) addresses the historical inefficiencies of manual university management.

1. Results: System Integration & Performance

The primary outcome of the project is a **Unified University Management & Student Support Platform** that consolidates fragmented processes into a single source of truth.

A. Modular Integration Success

- **Unified Ecosystem:** By merging Academic (Attendance), Hostel (Gatepass), and Administrative (Grievance) functions, the system eliminated data silos. A single user identity now flows seamlessly across all modules.
- **Role-Based Dashboard Efficiency:** We successfully implemented five distinct interfaces.
    - **Students** receive real-time updates on their attendance percentages.
    - **Faculty** can execute bulk attendance marking in seconds.
    - **Admins** gain a bird's-eye view of institutional health through an analytics-driven sidebar.

B. Automation & Workflow Optimization

- **The "Zero-Leak" Gatepass System:** The manual paperwork for hostel exits was replaced by a digital chain. The result is a transparent, timestamped log that prevents unauthorized exits and keeps parents informed.
- **Proactive Attendance Monitoring:** Instead of manual calculations at the end of a semester, the system uses **Cron Jobs** to trigger automatic alerts. This allows students to take corrective action before falling below the mandatory threshold ($< 75\%$).

2. Observations & Key Findings

During the development and testing phases, several critical insights were noted regarding the architecture and user experience:

- **The Power of Frontend-First Development:** By building the **React.js** UI components and **Figma** wireframes before the backend logic, the team was able to validate the user journey early. This reduced "UI Friction" and ensured the final product was intuitive for non-technical users like parents.
- **Maintainability via Modularization:** Because the system was built modularly (e.g., the Grievance module is independent of the Attendance module), bugs in one area did not crash the entire platform. This proves that the **Microservices-lite** approach is ideal for expanding university needs.
- **Real-Time Data Gratification:** Users reported higher satisfaction when they could see "Live Status" updates (e.g., "Gatepass: Pending Warden Approval") rather than waiting for manual verbal confirmation.

3. Challenges Faced & Technical Resolutions

No complex system is built without hurdles; the following technical challenges were overcome:

- **Role-Based Complexity:** Managing the permission matrix for five roles was complex. We resolved this by centralizing the **RBAC (Role-Based Access Control)** logic in a single middleware layer, ensuring that permissions are checked at the API level, not just the UI level.
- **State Synchronization:** Keeping the frontend and backend in sync during high-frequency updates (like bulk attendance) required optimized **REST API** patterns and efficient state management in React to prevent "stale data" from being displayed.
- **UI Scalability:** Designing a sidebar that could hold 10+ modules without looking cluttered was solved using **Tailwind CSS** drawer components and collapsible menus, maintaining a clean aesthetic.

4. Discussion: The Impact of Digitization

The success of EduSphere demonstrates that **digitization is not just about moving paper to a screen—it's about re-engineering the workflow.**

- **Transparency & Accountability:** By digitizing grievances and gatepasses, the system creates an audit trail. This reduces bias and ensures that every request is handled within a professional timeframe.
- **Communication Gap Bridging:** The inclusion of the **Parent** and **Warden** roles directly into the software ecosystem closes the communication gap that often leads to safety concerns in hostel management.
- **Future Scalability:** The current architecture is "Plugin-Ready." In the future, modules for **Fee Payment**, **Library Management**, or **Alumni Tracking** can be plugged into the existing JWT-secured framework with minimal restructuring.

14. Conclusion &Future Work

The **Conclusion and Future Work** section serves as the final synthesis of the project's success while providing a strategic roadmap for its evolution. It transitions **EduSphere** from a functional prototype to a vision of a "Smart Campus" ecosystem.

1. Final Conclusion: The EduSphere Impact

The development of EduSphere successfully addresses the fragmentation often found in academic administration. By consolidating disparate workflows into a **Unified University Management & Student Support Platform**, the project achieves three core pillars:

A. Operational Efficiency & Automation

The transition from manual, paper-based tracking to an automated **Node.js/Django backend** has eliminated redundant data entry. Features like the **Cron Job-based attendance monitoring** ensure that administrative tasks that previously took hours are now executed in milliseconds, reducing human error and oversight.

B. Stakeholder Synchronicity

EduSphere acts as a digital bridge between **Students, Faculty, Admins, Parents, and Wardens**. The implementation of **Role-Based Access Control (RBAC)** ensures that while the data is centralized, the visibility is personalized. This transparency builds trust, especially in sensitive areas like hostel gatepass management and grievance resolution.

C. Technical Robustness

Using a **React-Vite** frontend and **JWT-based security**, the system provides a high-performance, secure environment. The project proves that a **Modular-Iterative methodology** allows for a complex system to be built reliably, ensuring that the platform is ready for immediate deployment in a university setting.

2. Future Work: Toward a Smart Campus

While the current version of EduSphere provides a solid foundation, the modular architecture allows for the integration of "Next-Gen" features to make the system more intelligent and accessible.

A. Intelligence & Automation

- **AI-Powered Performance Analytics:** Utilizing machine learning models to analyze attendance and marks data to predict student outcomes, allowing faculty to intervene early for at-risk students.
- **NLP-Integrated Chatbot:** Enhancing the grievance module with a **Natural Language Processing (NLP)** bot that can resolve common student queries (e.g., "How do I apply for a gatepass?") without administrative intervention.

B. Hardware & Real-Time Integration

- **Biometric & IoT Synchronization:** Moving beyond manual digital logs by integrating **RFID or Fingerprint scanners** at hostel gates. This data would sync directly with the EduSphere database via APIs.
- **WebSockets for Instant Alerts:** Implementing **Socket.io** to provide real-time push notifications. Instead of checking a dashboard, parents and students would receive instant alerts for gatepass approvals or emergency notices.

C. Accessibility & Advanced Visualization

- **Mobile-First Expansion:** Developing a dedicated **React Native** or Flutter application to provide a native experience, leveraging mobile features like camera-based QR scanning for gate entries.
- **Data Science Dashboards:** Integrating advanced data visualization tools (like D3.js or PowerBI embeds) to provide university deans with high-level trends regarding institutional growth, faculty performance, and student satisfaction.

15. REFERENCES

[1]. Laudon, K. C., & Laudon, J. P. (2020).

Management Information Systems: Managing the Digital Firm. Pearson Education.

[2]. O'Brien, J. A., & Marakas, G. M. (2019).

Management Information Systems. McGraw-Hill Education.

[3]. Pressman, R. S., & Maxim, B. R. (2020).

Software Engineering: A Practitioner's Approach. McGraw-Hill.

[4]. Sommerville, I. (2016).

Software Engineering. Pearson Education.

[5]. UNESCO (2021).

Digital Transformation of Higher Education Institutions. UNESCO Publishing.

[6]. UGC (University Grants Commission), India.

Guidelines on Digital Initiatives in Higher Education.

[7]. Aljawarneh, S. A. (2017).

"Reviewing and Exploring Innovative University Management Systems."

International Journal of Advanced Computer Science and Applications (IJACSA).

ANNEXURE I:

Plagiarism Declaration Certificate

Title of Work: EduSphere
Submitted By: Techno_Crafts
Institution: K. R. Mangalam University, Gurugram
Department: SOET
Date of Submission: 06.04.2026

I hereby declare that the work **EduSphere** entitled submitted for academic evaluation and research purposes is my original work. I confirm that:

- I have acknowledged and properly cited all sources, references, and data included in this work.
- This work does not contain any material previously published, written, or prepared by another person, except where due acknowledgment has been made.
- I understand that plagiarism is an academic offense and a violation of research ethics. Any breach of this declaration may lead to disciplinary action as per university policies.
- I have used appropriate referencing techniques and maintained academic integrity throughout this work.

I affirm that the submitted work has normal plagiarism less than 10% and free from AI content.

Signature of Student: _____
Date: 06.04.2026

ANNEXURE II

PLAGIARISM SCORE
ORIGINALITY SCORE
95.8%
100
50
0
Estimated Similarity (~4.2%)
Originality Score (~95.8%)
INTERPRETATION
Very Low duplication
Codebase is largely original
Minor repetition likely due to:
• Boilerplate code (React components, configs)
• Reusable utilities
• Framework-generated files
Report generated: 2024-06-11 10:30 AM
Report generated: 2024-06-11 10:30 AM
CodeGuard

ANNEXURE III

CODE IMPLEMENTATION

The coding part in below with specific modules discussed above.

Index.html

```
<!doctype html>

<html lang="en">

  <head>

    <meta charset="UTF-8" />

    <link rel="icon" type="image/svg+xml" href="/vite.svg" />

    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <title>EduSphere</title>

    <script type="module" crossorigin src="/assets/index-bb7lA5i3.js"></script>

    <link rel="stylesheet" crossorigin href="/assets/index-CPwBfgLa.css">

  </head>

  <body>

    <div id="root"></div>

  </body>

</html>
```

Timetable.js

```
const mongoose = require("mongoose");
const timetableEntrySchema = new mongoose.Schema(
    {
        course: {
```

```
        type: mongoose.Schema.Types.ObjectId,
        ref: "Course",
        required: true,
    },
    department: {
        type: String,
        required: true,
    },
    semester: {
        type: Number,
        required: true,
    },
    day: {
        type: String,
        enum: ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"],
        required: true,
    },
    startTime: {
        type: String, // e.g., "10:00 AM"
        required: true,
    },
    endTime: {
        type: String, // e.g., "11:00 AM"
        required: true,
    },
    roomContext: {
        type: String, // e.g. "Room 402" or "Lab 1"
    },
},
{
```

```
        timestamps: true,
    }
);
// exports module
module.exports = mongoose.model("Timetable", timetableEntrySchema);
```

Enrollment.js

```
const mongoose = require("mongoose");
const enrollmentSchema = new mongoose.Schema(
    {
        student: {
            type: mongoose.Schema.Types.ObjectId,
            ref: "User",
            required: true,
        },
        course: {
            type: mongoose.Schema.Types.ObjectId,
            ref: "Course",
            required: true,
        },
        status: {
            type: String,
            enum: ["enrolled", "dropped"],
            default: "enrolled",
        },
    },
    {
        timestamps: true,
    }
);
```

```
// Prevent duplicate enrollments
enrollmentSchema.index({ student: 1, course: 1 }, { unique: true });
module.exports = mongoose.model("Enrollment", enrollmentSchema);
```